MAHARISHI MARKANDESHWAR (DEEMED TO BE UNIVERSITY)

Mullana-Ambala, Haryana

(Established under Section 3 of UGC Act, 1956)

(Accredited by NAAC with Grade 'A++')

(Artificial Neural Network Lab)

BCSE-821L

SUBMITTED BY:

NAME :- ANKIT RAJ

ROLL.NO :- 11233036

SECTION :- 6G

SUBMITTED TO:

Dr. Aayush Shrivastava

PROFESSOR

(CSE DEPARTMENT)

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

M.M. ENGINEERING COLLEGE

MAHARISHI MARKANDESHWAR (DEEMED TO BE UNIVERSITY)

MULLANA,AMBALA,HARYANA

2023-2027

Experiment Number :- 01
Aim:- Introduction to Neural Network Fundamentals and Perceptron.
Software Used:-

1. **Python:-** Programming Language
2. **NumPy:-** For numerical computations
3. **Matplotlib:-** For plotting graphs
4. **TensorFlow / Keras:-** For building neural network models
5. **Jupyter Notebook/ Google colab/ VS Code:-** Development environment

Library Used:-
NumPy, Matplotlib, TensorFlow and Keras
References:-

1. Frank Rosenblatt (1958), *The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain*, Psychological Review.
2. Simon Haykin, *Neural Networks and Learning Machines*, Pearson Education.
3. Christopher M. Bishop, *Pattern Recognition and Machine Learning*, Springer.
4. Ian Goodfellow, Yoshua Bengio, Aaron Courville, *Deep Learning*, MIT Press.
5. Official Documentation:
    - NumPy Documentation – https://numpy.org
    - Matplotlib Documentation – https://matplotlib.org
    - TensorFlow Documentation – https://www.tensorflow.org

Methodology:-

1. First, we studied the basic concepts of Neural Networks and Perceptron.
2. Then, we implemented a simple neuron model using inputs, weights, bias, and activation function.
3. After that, we applied the Perceptron learning algorithm to train the model.
4. We tested the perceptron on logical gates like AND and OR and observed the results.
5. We implemented a multi-layer neural network to solve complex problems like XOR and MNIST digit classification.
6. Finally, we compared different activation functions (Sigmoid, Tanh, ReLU, Leaky ReLU) using graphs.

INTRODUCTION TO NEURAL NETWORK FUNDAMENTALS:-
Artificial Neural Networks (ANNs) are computational models inspired by the human brain. They are used to solve problems like classification, prediction, pattern recognition, and decision making.

A neural network is made up of small processing units called **neurons**. Each neuron receives input values, multiplies them with weights, adds a bias, and then applies an activation function to produce an output.
The basic structure of a neural network includes:

- **Input Layer** – Takes input data
- **Hidden Layer(s)** – Processes the data
- **Output Layer** – Produces the final result

The output of a neuron is calculated using:

$$Y = f(\sum w_i x_i + b)$$

Where:

- x_i= Input values
- w_i= Weights
- b= Bias
- f= Activation function

Neural networks learn by adjusting weights using learning algorithms such as the Perceptron learning rule or Backpropagation.
These models are widely used in image recognition, speech recognition, medical diagnosis, and many real-world applications.

INTRODUCTION TO PERCEPTRON NETWORK FUNDAMENTALS:-

The Perceptron is the simplest type of Artificial Neural Network. It was developed by **Frank Rosenblatt in 1958**. It is mainly used for solving binary classification problems, where the output is either 0 or 1.
A perceptron consists of:

- Input values
- Weights
- Bias
- Activation function (usually Step function)

Each input is multiplied by a weight, and all values are added together with a bias. The result is then passed through an activation function to produce the final output.
The mathematical model of a perceptron is:

$$Y = f(\sum w_i x_i + b)$$

Where:

- x_i= Input values
- w_i= Weights
- b= Bias
- f= Step activation function
- Y= Output

The perceptron learns by updating weights using the Perceptron learning rule:

$$w_{new} = w_{old} + \eta(t - y)x$$

It works only for linearly separable problems such as AND and OR gates. It cannot solve non-linearly separable problems like XOR.
The perceptron forms the foundation of modern deep learning models.

Experiment Number :- 02
Aim:- Implement a basic neuron model and the Perceptron learning algorithm.
Software Used:-

- **Python 3.x**
- **NumPy library** (for numerical calculations)
- **Jupyter Notebook / Google Colab / VS Code** (any Python IDE)

Library Used:-
NumPy
References:-

1. **Neural Networks and Learning Machines**
   Author: Simon Haykin
   Publisher: Pearson Education
2. **Artificial Intelligence: A Modern Approach**
   Authors: Stuart Russell and Peter Norvig
   Publisher: Pearson
3. NumPy Documentation –
   NumPy
   Website: https://numpy.org

Methodology:-

1. **Initialize values** – Set input values, target output, weights, bias, and learning rate.
2. **Calculate net input** – Multiply inputs with weights and add bias.
3. **Apply activation function** – If value ≥ 0 output is 1, otherwise output is 0.
4. **Calculate error** – Find the difference between target output and predicted output.
5. **Update weights and bias** – Adjust weights and bias using the error and repeat the process until correct output is obtained.

IMPLEMENTATION OF BASIC NEURON MODEL USING PYTHON:-
Source code:-

```
import numpy as np
# Inputs and targets
X = np.array([[0,0],
              [0,1],
              [1,0],
              [1,1]])
T = np.array([0, 0, 0, 1])   # AND gate
```

```
# Initial weights and bias
w = np.array([0.0, 0.0])
b = 0.0
alpha = 0.1
# Training with iterations
for epoch in range(5):
    print("Iteration:", epoch + 1)
    for i in range(len(X)):
        v = np.dot(w, X[i]) + b
        if v >= 0:
            y = 1
        else:
            y = 0
        error = T[i] - y
        w = w + alpha * error * X[i]
        b = b + alpha * error
        print("Input:", X[i], "Output:", y, "Error:", error)
    print("Weights:", w, "Bias:", b)
```

Output:-

```
PS C:\Users\saifa\6th Sem\Lab\ANN\File> & "C:/Program Files/Python312/pyt
Iteration: 1
Input: [0 0] Output: 1 Error: -1
Input: [0 1] Output: 0 Error: 0
Input: [1 0] Output: 0 Error: 0
Input: [1 1] Output: 0 Error: 1
Weights: [0.1 0.1] Bias: 0.0

Iteration: 2
Input: [0 0] Output: 1 Error: -1
Input: [0 1] Output: 1 Error: -1
Input: [1 0] Output: 0 Error: 0
Input: [1 1] Output: 0 Error: 1
Weights: [0.2 0.1] Bias: -0.1

Iteration: 3
Input: [0 0] Output: 0 Error: 0
Input: [0 1] Output: 1 Error: -1
Input: [1 0] Output: 1 Error: -1
Input: [1 1] Output: 0 Error: 1
Weights: [0.2 0.1] Bias: -0.20000000000000004

Iteration: 4
Input: [0 0] Output: 0 Error: 0
Input: [0 1] Output: 0 Error: 0
Input: [1 0] Output: 0 Error: 0
Input: [1 1] Output: 1 Error: 0
Weights: [0.2 0.1] Bias: -0.20000000000000004

Iteration: 5
Input: [0 0] Output: 0 Error: 0
Input: [0 1] Output: 0 Error: 0
Input: [1 0] Output: 0 Error: 0
Input: [1 1] Output: 1 Error: 0
Weights: [0.2 0.1] Bias: -0.20000000000000004
```

IMPLEMENTATION OF BASIC PERCEPTRON MODEL(AND GATE) USING PYTHON:-

```
import numpy as np
# Inputs and targets (AND gate)
X = np.array([[0,0],
              [0,1],
              [1,0],
              [1,1]])
T = np.array([0, 0, 0, 1])
# Initial weights and bias
w = np.array([0.0, 0.0])
b = 0.0
alpha = 0.1
# Training
for epoch in range(5):
    print("Iteration:", epoch + 1)
    for i in range(len(X)):
        v = np.dot(w, X[i]) + b
        if v >= 0:
            y = 1
        else:
            y = 0
        error = T[i] - y
        w = w + alpha * error * X[i]
        b = b + alpha * error
        print("Input:", X[i], "Target:", T[i], "Output:", y, "Error:", error)
    print("Updated Weights:", w, "Updated Bias:", b)
```

Output:-

```
PS C:\Users\saifa\6th Sem\Lab\ANN\File> & "C:/Program Files/Python312/python.exe
Iteration: 1
Input: [0 0] Target: 0 Output: 1 Error: -1
Input: [0 1] Target: 0 Output: 0 Error: 0
Input: [1 0] Target: 0 Output: 0 Error: 0
Input: [1 1] Target: 1 Output: 0 Error: 1
Updated Weights: [0.1 0.1] Updated Bias: 0.0

Iteration: 2
Input: [0 0] Target: 0 Output: 1 Error: -1
Input: [0 1] Target: 0 Output: 1 Error: -1
Input: [1 0] Target: 0 Output: 0 Error: 0
Input: [1 1] Target: 1 Output: 0 Error: 1
Updated Weights: [0.2 0.1] Updated Bias: -0.1

Iteration: 3
Input: [0 0] Target: 0 Output: 0 Error: 0
Input: [0 1] Target: 0 Output: 1 Error: -1
Input: [1 0] Target: 0 Output: 1 Error: -1
Input: [1 1] Target: 1 Output: 0 Error: 1
Updated Weights: [0.2 0.1] Updated Bias: -0.20000000000000004

Iteration: 4
Input: [0 0] Target: 0 Output: 0 Error: 0
Input: [0 1] Target: 0 Output: 0 Error: 0
Input: [1 0] Target: 0 Output: 0 Error: 0
Input: [1 1] Target: 1 Output: 1 Error: 0
Updated Weights: [0.2 0.1] Updated Bias: -0.20000000000000004

Iteration: 5
Input: [0 0] Target: 0 Output: 0 Error: 0
Input: [0 1] Target: 0 Output: 0 Error: 0
Input: [1 0] Target: 0 Output: 0 Error: 0
Input: [1 1] Target: 1 Output: 1 Error: 0
Updated Weights: [0.2 0.1] Updated Bias: -0.20000000000000004
```

Experiment Number :- 03

Aim:- Create a perceptron from scratch using Python to solve logical problems like AND, OR, XOR.

Software Used:-

- **Python 3.x**
- **NumPy library** (for numerical calculations)
- **Jupyter Notebook / Google Colab / VS Code** (any Python IDE)

Library Used:-

NumPy

References:-

1. **Neural Networks and Learning Machines**
   Author: Simon Haykin
   Publisher: Pearson Education
2. **Artificial Intelligence: A Modern Approach**
   Authors: Stuart Russell and Peter Norvig
   Publisher: Pearson
3. NumPy Documentation –
   NumPy
   Website: https://numpy.org

Methodology:-

1. **Initialize values** – Set input values, target output, weights, bias, and learning rate.
2. **Calculate net input** – Multiply inputs with weights and add bias.
3. **Apply activation function** – If value ≥ 0 output is 1, otherwise output is 0.
4. **Calculate error** – Find the difference between target output and predicted output.
5. **Update weights and bias** – Adjust weights and bias using the error and repeat the process until correct output is obtained.

Source Code:-

```
import numpy as np
# User selects gate
gate = input("Enter gate (AND / OR / XOR): ").upper()
X = np.array([[0,0],
              [0,1],
              [1,0],
              [1,1]])
```

```
if gate == "AND":
    T = np.array([0,0,0,1])
elif gate == "OR":
    T = np.array([0,1,1,1])
elif gate == "XOR":
    T = np.array([0,1,1,0])
else:
    print("Invalid Gate Selected")
    exit()
# Initialize weights and bias
w = np.array([0.0, 0.0])
b = 0.0
alpha = 0.1
epochs = 10
print("\nRunning Gate:", gate)
# Training
for epoch in range(epochs):
    print("Iteration:", epoch + 1)
    total_error = 0
    for i in range(len(X)):
        v = np.dot(w, X[i]) + b
        if v >= 0:
            y = 1
        else:
            y = 0
        error = T[i] - y
        total_error += abs(error)
        w = w + alpha * error * X[i]
        b = b + alpha * error
        print("Input:", X[i], "Target:", T[i], "Output:", y)
    print("Weights:", w, "Bias:", b)
    if total_error == 0:
        print("Training Complete")
        break
print("\nFinal Weights:", w)
print("Final Bias:", b)
```

OUTPUT:-

- For AND GATE:

```
PS C:\Users\saifa\6th Sem\Lab\ANN\File> & "C:/Program Files/Python
Enter gate (AND / OR / XOR): AND

Running Gate: AND
Iteration: 1
Input: [0 0] Target: 0 Output: 1
Input: [0 1] Target: 0 Output: 0
Input: [1 0] Target: 0 Output: 0
Input: [1 1] Target: 1 Output: 0
Weights: [0.1 0.1] Bias: 0.0
Iteration: 2
Input: [0 0] Target: 0 Output: 1
Input: [0 1] Target: 0 Output: 1
Input: [1 0] Target: 0 Output: 0
Input: [1 1] Target: 1 Output: 0
Weights: [0.2 0.1] Bias: -0.1
Iteration: 3
Input: [0 0] Target: 0 Output: 0
Input: [0 1] Target: 0 Output: 1
Input: [1 0] Target: 0 Output: 1
Input: [1 1] Target: 1 Output: 0
Weights: [0.2 0.1] Bias: -0.20000000000000004
Iteration: 4
Input: [0 0] Target: 0 Output: 0
Input: [0 1] Target: 0 Output: 0
Input: [1 0] Target: 0 Output: 0
Input: [1 1] Target: 1 Output: 1
Weights: [0.2 0.1] Bias: -0.20000000000000004
Training Complete

Final Weights: [0.2 0.1]
Final Bias: -0.20000000000000004
PS C:\Users\saifa\6th Sem\Lab\ANN\File>
```

- For OR GATE

```
PS C:\Users\saifa\6th Sem\Lab\ANN\File> & "C:/Program Files/
Enter gate (AND / OR / XOR): OR

Running Gate: OR
Iteration: 1
Input: [0 0] Target: 0 Output: 1
Input: [0 1] Target: 1 Output: 0
Input: [1 0] Target: 1 Output: 1
Input: [1 1] Target: 1 Output: 1
Weights: [0.  0.1] Bias: 0.0
Iteration: 2
Input: [0 0] Target: 0 Output: 1
Input: [0 1] Target: 1 Output: 1
Input: [1 0] Target: 1 Output: 0
Input: [1 1] Target: 1 Output: 1
Weights: [0.1 0.1] Bias: 0.0
Iteration: 3
Input: [0 0] Target: 0 Output: 1
Input: [0 1] Target: 1 Output: 1
Input: [1 0] Target: 1 Output: 1
Input: [1 1] Target: 1 Output: 1
Weights: [0.1 0.1] Bias: -0.1
Iteration: 4
Input: [0 0] Target: 0 Output: 0
Input: [0 1] Target: 1 Output: 1
Input: [1 0] Target: 1 Output: 1
Input: [1 1] Target: 1 Output: 1
Weights: [0.1 0.1] Bias: -0.1
Training Complete

Final Weights: [0.1 0.1]
Final Bias: -0.1
PS C:\Users\saifa\6th Sem\Lab\ANN\File>
```

- For XOR Gate

```
PS C:\Users\saifa\6th Sem\Lab\ANN\File> &
Enter gate (AND / OR / XOR): XOR

Running Gate: XOR
Iteration: 1
Input: [0 0] Target: 0 Output: 1
Input: [0 1] Target: 1 Output: 0
Input: [1 0] Target: 1 Output: 1
Input: [1 1] Target: 0 Output: 1
Weights: [-0.1  0. ] Bias: -0.1
Iteration: 2
Input: [0 0] Target: 0 Output: 0
Input: [0 1] Target: 1 Output: 0
Input: [1 0] Target: 1 Output: 0
Input: [1 1] Target: 0 Output: 1
Weights: [-0.1  0. ] Bias: 0.0
Iteration: 3
Input: [0 0] Target: 0 Output: 1
Input: [0 1] Target: 1 Output: 0
Input: [1 0] Target: 1 Output: 0
Input: [1 1] Target: 0 Output: 1
Weights: [-0.1  0. ] Bias: 0.0
Iteration: 4
Input: [0 0] Target: 0 Output: 1
Input: [0 1] Target: 1 Output: 0
Input: [1 0] Target: 1 Output: 0
Input: [1 1] Target: 0 Output: 1
Weights: [-0.1  0. ] Bias: 0.0
Iteration: 5
Input: [0 0] Target: 0 Output: 1
Input: [0 1] Target: 1 Output: 0
Input: [1 0] Target: 1 Output: 0
Input: [1 1] Target: 0 Output: 1
Weights: [-0.1  0. ] Bias: 0.0
Iteration: 6
Input: [0 0] Target: 0 Output: 1
Input: [0 1] Target: 1 Output: 0
Input: [1 0] Target: 1 Output: 0
Input: [1 1] Target: 0 Output: 1
Weights: [-0.1  0. ] Bias: 0.0
```

Experiment Number :- 04
Aim:- Use the MNIST dataset to build a digit classifier with multiple hidden layers.
Software Used:-
- **Tensorflow**
- **Keras**
- **Jupyter Notebook / Google Colab / VS Code** (any Python IDE)

Library Used:-
NumPy
References:-
1. **MNIST**
   Authors: Yann LeCun, Corinna Cortes, Christopher J. C. Burges
   Title: *The MNIST Database of Handwritten Digits*
   Source: AT&T Labs / Yann LeCun's Website
2. **Deep Learning**
   Authors: Ian Goodfellow, Yoshua Bengio, Aaron Courville
   Publisher: MIT Press

Methodology:-
1. **Load MNIST data** – Take handwritten digit images for training and testing.
2. **Prepare the data** – Convert images into numbers and scale values between 0 and 1.
3. **Create the model** – Build a neural network with input layer, two or more hidden layers, and one output layer (0–9 digits).
4. **Train the model** – Give training data to the model and let it learn by adjusting weights for several iterations (epochs).
5. **Test the model** – Check accuracy using test data to see how correctly it predicts digits.

Source Code:-
```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
# Load MNIST dataset
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
# Normalize data (0-255 → 0-1)
x_train = x_train / 255.0
x_test = x_test / 255.0
```

```
# Build model
model = keras.Sequential([
    layers.Flatten(input_shape=(28, 28)),
    layers.Dense(128, activation='relu'),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])
# Compile model
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
# Train model
model.fit(x_train, y_train, epochs=5)
# Evaluate model
test_loss, test_acc = model.evaluate(x_test, y_test)
print("Test Accuracy:", test_acc)
```

OUTPUT:-

```
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 FMA, in other o
Epoch 1/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 3s 1ms/step - accuracy: 0.8748 - loss: 0.4306
Epoch 2/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 3s 1ms/step - accuracy: 0.9693 - loss: 0.1027
Epoch 3/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9785 - loss: 0.0676
Epoch 4/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9846 - loss: 0.0504
Epoch 5/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 3s 1ms/step - accuracy: 0.9889 - loss: 0.0369
313/313 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9730 - loss: 0.0938
Test Accuracy: 0.9771000146865845
PS C:\Users\saifa\6th Sem\Lab\ANN\File>
```

Experiment Number :- 05
Aim:- Implement a back propagation algorithm using only NumPy without ML libraries.
Software Used:-

- **Python 3.x**
- **NumPy library** (for numerical calculations)
- **Jupyter Notebook / Google Colab / VS Code** (any Python IDE)

Library Used:-
NumPy
References:-

1. **Neural Networks and Learning Machines**
   Author: Simon Haykin
   Publisher: Pearson Education
   (Explains backpropagation algorithm in detail.)
2. **Deep Learning**
   Authors: Ian Goodfellow, Yoshua Bengio, Aaron Courville
   Publisher: MIT Press
   (Explains gradient descent and backpropagation clearly.)

Methodology:-

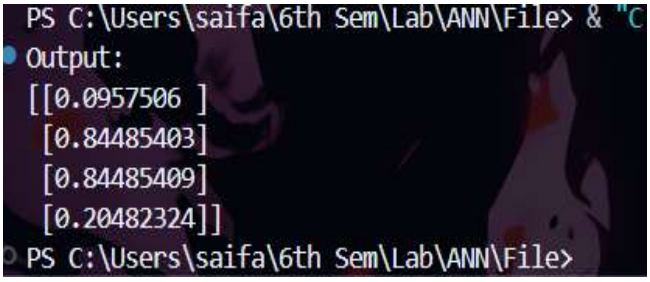
1. **Initialize values** – Set input, output, weights, bias, and learning rate.
2. **Forward pass** – Multiply inputs with weights, add bias, and calculate output using activation function (like sigmoid).
3. **Find error** – Calculate the difference between target value and predicted output.
4. **Backward pass** – Compute gradients (derivatives) and send error back to update hidden layer weights.
5. **Update weights** – Adjust weights using learning rate and repeat the process for many iterations until error becomes small.

Source Code:-
Simple Back Propagation Code(XOR Problem)

```
import numpy as np
# Input (XOR)
X = np.array([[0,0],
              [0,1],
              [1,0],
              [1,1]])
```

```
y = np.array([[0],[1],[1],[0]])
# Sigmoid function
def sigmoid(x):
    return 1 / (1 + np.exp(-x))
# Derivative
def d_sigmoid(x):
    return x * (1 - x)
# Random weights
np.random.seed(0)
w1 = np.random.rand(2,2)
w2 = np.random.rand(2,1)
lr = 0.5
# Training
for i in range(5000):
    # Forward
    h = sigmoid(np.dot(X, w1))
    o = sigmoid(np.dot(h, w2))
    # Error
    error = y - o
    # Backward
    d_o = error * d_sigmoid(o)
    d_h = np.dot(d_o, w2.T) * d_sigmoid(h)
    # Update
    w2 += lr * np.dot(h.T, d_o)
    w1 += lr * np.dot(X.T, d_h)
# Final output
print("Output:")
print(o)
```

OUTPUT:-



Experiment Number :- 06

Aim:- Plot and compare Sigmoid, Tanh, ReLU, and LeakyReLU with real input data.

Software Used:-

- **Python 3.x**
- **Jupyter Notebook / Google Colab / VS Code** (any Python IDE)

Library Used:-

1. **NumPy** – For numerical calculations and creating input values.
2. **Matplotlib** – For plotting graphs.

References:-

1. **Deep Learning**
   Authors: Ian Goodfellow, Yoshua Bengio, Aaron Courville
   Publisher: MIT Press
2. **Neural Networks and Learning Machines**
   Author: Simon Haykin
   Publisher: Pearson Education

Methodology:-

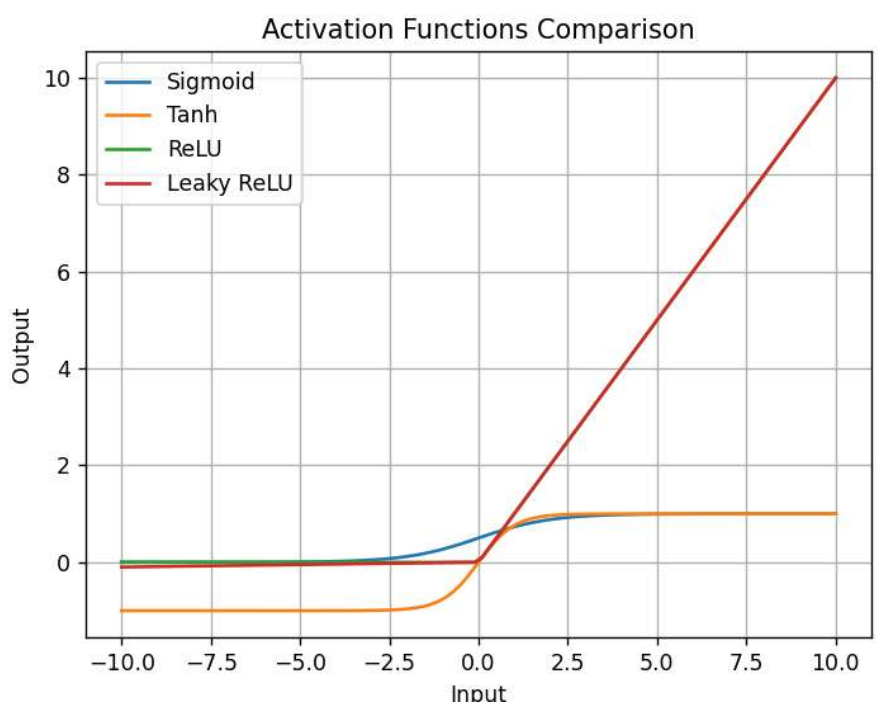
1. **Create Input Data** – Generate real input values (for example from -10 to 10).
2. **Define Activation Functions** – Write formulas for Sigmoid, Tanh, ReLU, and Leaky ReLU.
3. **Apply Functions** – Pass the input values into each activation function.
4. **Plot the Graphs** – Use matplotlib to plot all activation functions.
5. **Compare Results** – Observe differences in output shape and behavior.

Source Code:-

```
import numpy as np
import matplotlib.pyplot as plt
# Create input values
x = np.linspace(-10, 10, 100)
# Activation functions
def sigmoid(x):
    return 1 / (1 + np.exp(-x))
def tanh(x):
    return np.tanh(x)
def relu(x):
    return np.maximum(0, x)
```

```
def leaky_relu(x):
    return np.where(x > 0, x, 0.01 * x)
# Plot functions
plt.plot(x, sigmoid(x), label="Sigmoid")
plt.plot(x, tanh(x), label="Tanh")
plt.plot(x, relu(x), label="ReLU")
plt.plot(x, leaky_relu(x), label="Leaky ReLU")

plt.title("Activation Functions Comparison")
plt.xlabel("Input")
plt.ylabel("Output")
plt.legend()
plt.grid(True)
plt.show()
```

OUTPUT:-



Experiment Number: 07

Aim: Learn to load, preprocess, and evaluate neural network models on standard datasets. (Calculate and interpret evaluation metrics such as accuracy, precision, recall, F1-score for classification; MSE, RMSE for regression)

Software Used

- Google Colab or Jupyter Notebook or VS Code

Libraries Used

- NumPy
- Matplotlib
- Scikit-learn
- TensorFlow / Keras

Theory

Evaluating a neural network model requires measuring how well its predictions match the true labels. Different metrics are used depending on whether the task is classification or regression.

A. Classification Metrics

Accuracy: Ratio of correctly predicted samples to total samples.
Accuracy = (TP + TN) / (TP + TN + FP + FN)
Precision: Out of all positive predictions, how many were actually positive.
Precision = TP / (TP + FP)
Recall (Sensitivity): Out of all actual positives, how many were correctly identified.
Recall = TP / (TP + FN)
F1-Score: Harmonic mean of Precision and Recall. Best when classes are imbalanced.
F1 = 2 × (Precision × Recall) / (Precision + Recall)

B. Regression Metrics

MSE (Mean Squared Error): Average of squared differences between actual and predicted values.
$MSE = (1/n) \Sigma (y_i - \hat{y}_i)^2$
RMSE (Root Mean Squared Error): Square root of MSE; in the same unit as the target variable.
$RMSE = \sqrt{MSE}$
R^2 Score: Proportion of variance explained by the model. Ranges from 0 to 1; higher is better.

Methodology

1. Open Google Colab / Jupyter Notebook / VS Code.
2. Create a new Python notebook (.ipynb).
3. Import required libraries: NumPy, Matplotlib, Scikit-learn, TensorFlow/Keras.
4. Load standard datasets:
   a. Breast Cancer dataset (classification)
   b. California Housing dataset (regression)
5. Preprocess data: train-test split and StandardScaler normalization.
6. Build neural network models using Keras Sequential API.
7. Train the models on the training set.
8. Generate predictions on the test set.
9. Calculate classification metrics: Accuracy, Precision, Recall, F1-Score.
10. Calculate regression metrics: MSE, RMSE, R^2 Score.
11. Plot Confusion Matrix for classification and Actual vs Predicted for regression.
12. Observe and interpret the results.

Code:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer, fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.metrics import confusion_matrix, classification_report
from sklearn.metrics import mean_squared_error, r2_score
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# PART A: CLASSIFICATION

# Load dataset
data = load_breast_cancer()
X = data.data
y = data.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# Scale data
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

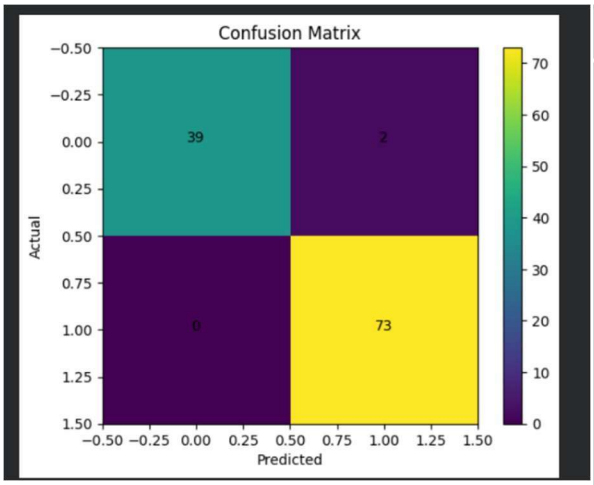
```
# Create model
model = Sequential()
model.add(Dense(64, activation='relu', input_shape=(X_train.shape[1],)))
model.add(Dense(32, activation='relu'))
model.add(Dense(1, activation='sigmoid'))

# Compile model
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# Train model
model.fit(X_train, y_train, epochs=50, batch_size=16, verbose=0)

# Prediction
y_pred = model.predict(X_test)
y_pred = (y_pred > 0.5)

# Evaluation
print("=== Classification ===")
print("Accuracy :", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall   :", recall_score(y_test, y_pred))
print("F1 Score :", f1_score(y_test, y_pred))

print("\nReport:\n", classification_report(y_test, y_pred))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

plt.imshow(cm)
plt.title("Confusion Matrix")
plt.colorbar()
plt.xlabel("Predicted")
plt.ylabel("Actual")

for i in range(2):
    for j in range(2):
        plt.text(j, i, cm[i][j], ha='center')

plt.show()
```

 # PART B: REGRESSION

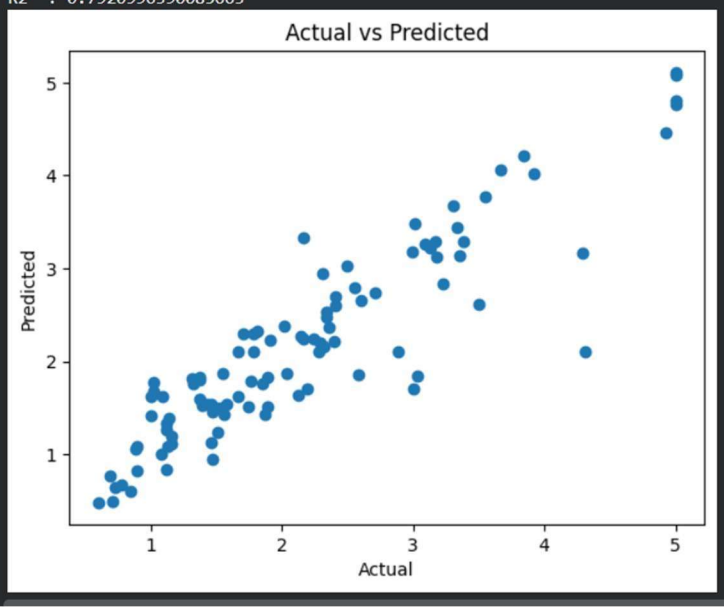
```
# Load dataset
house = fetch_california_housing()
X = house.data
y = house.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# Scale data
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Create model
model = Sequential()
model.add(Dense(64, activation='relu', input_shape=(X_train.shape[1],)))
model.add(Dense(32, activation='relu'))
model.add(Dense(1))

# Compile and train
model.compile(optimizer='adam', loss='mse')
model.fit(X_train, y_train, epochs=50, batch_size=32, verbose=0)

# Prediction
y_pred = model.predict(X_test)

# Evaluation
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("\n=== Regression ===")
print("MSE :", mse)
print("RMSE:", rmse)
print("R2  :", r2)

# Plot graph
plt.scatter(y_test[:100], y_pred[:100])
plt.xlabel("Actual")
plt.ylabel("Predicted")
plt.title("Actual vs Predicted")
plt.show()
```

Output:

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step
=== Classification ===
Accuracy : 0.956140350877193
Precision: 0.9615384615384616
Recall : 0.974025974025974
F1 Score : 0.967741935483871

Report:
              precision    recall  f1-score   support

           0       0.94      0.92      0.93        37
           1       0.96      0.97      0.97        77

    accuracy                           0.96       114
   macro avg       0.95      0.95      0.95       114
weighted avg       0.96      0.96      0.96       114
```




```
=== Regression ===
MSE : 0.27058662531691574
RMSE: 0.5201794164679296
R2  : 0.7920996390085663
```



Experiment Number: 08

Aim: Create a CNN for image classification using CIFAR-10 or Fashion-MNIST.

Software Used

- Google Colab / Jupyter Notebook / VS Code

Libraries Used

- NumPy
- Matplotlib
- TensorFlow / Keras

Methodology

1. Import required libraries
2. Load Fashion-MNIST dataset
3. Normalize pixel values (0–255 → 0–1)
4. Reshape data for CNN input
5. Build CNN model:
    - Convolution layers
    - Pooling layers
    - Fully connected layers
6. Compile the model
7. Train the model
8. Evaluate performance
9. Predict sample images

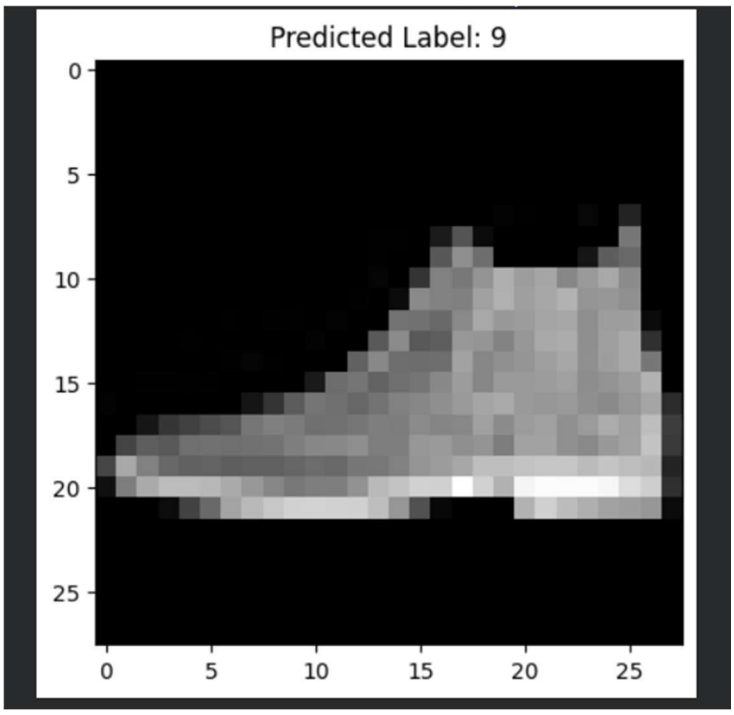
Code:

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import fashion_mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

# Step 1: Load dataset
(X_train, y_train), (X_test, y_test) = fashion_mnist.load_data()

# Step 2: Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0
```

```
# Step 3: Reshape for CNN (add channel dimension)
X_train = X_train.reshape(-1, 28, 28, 1)
X_test = X_test.reshape(-1, 28, 28, 1)

# Step 4: Build CNN model
model = Sequential()

model.add(Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)))
model.add(MaxPooling2D((2,2)))

model.add(Conv2D(64, (3,3), activation='relu'))
model.add(MaxPooling2D((2,2)))

model.add(Flatten())

model.add(Dense(64, activation='relu'))
model.add(Dense(10, activation='softmax'))

# Step 5: Compile model
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# Step 6: Train model
model.fit(X_train, y_train, epochs=5)

# Step 7: Evaluate model
loss, accuracy = model.evaluate(X_test, y_test)

print("Test Accuracy:", accuracy)

# Step 8: Predict and display sample
y_pred = model.predict(X_test)

plt.imshow(X_test[0].reshape(28,28), cmap='gray')
plt.title("Predicted Label: " + str(np.argmax(y_pred[0])))
plt.show()
```

Output:

```
Epoch 1/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 9s 3ms/step - accuracy: 0.8350 - loss: 0.4577
Epoch 2/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.8859 - loss: 0.3132
Epoch 3/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.9018 - loss: 0.2677
Epoch 4/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9131 - loss: 0.2354
Epoch 5/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.9221 - loss: 0.2093
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - accuracy: 0.9044 - loss: 0.2641
Test Accuracy: 0.9043999910354614
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step
```



Experiment Number: 09

Aim: To apply LSTM/GRU networks for sequence prediction using a simple time series dataset.

Software Used

- Google Colab / Jupyter Notebook / VS Code

Libraries Used

- NumPy
- Matplotlib
- TensorFlow / Keras

Methodology

1. Import required libraries
2. Create a synthetic time series dataset
3. Normalize the data
4. Prepare sequences (input-output pairs)
5. Build LSTM/GRU model
6. Train the model
7. Predict future values
8. Plot actual vs predicted values

Code:

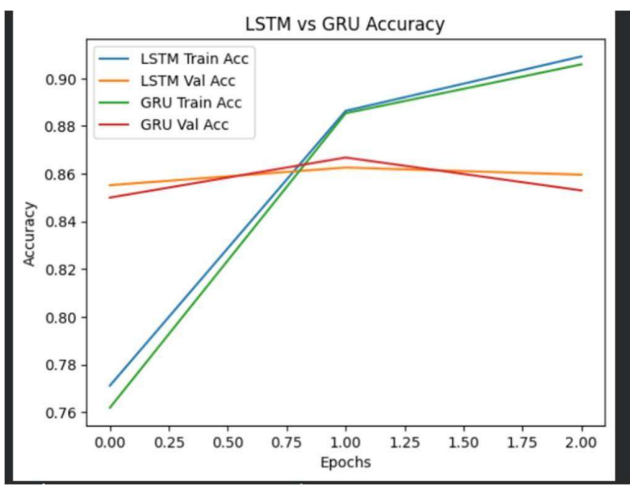
```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import imdb
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, GRU, Dense
from tensorflow.keras.preprocessing.sequence import pad_sequences

# Step 1: Load dataset
(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=5000)

# Step 2: Padding
X_train = pad_sequences(X_train, maxlen=200)
X_test = pad_sequences(X_test, maxlen=200)

#  LSTM MODEL
print("\n--- LSTM Model ---")
lstm_model = Sequential()
```

```
lstm_model.add(Embedding(input_dim=5000, output_dim=32, input_length=200))
lstm_model.add(LSTM(32))
lstm_model.add(Dense(1, activation='sigmoid'))

lstm_model.compile(optimizer='adam',
                   loss='binary_crossentropy',
                   metrics=['accuracy'])
# Train LSTM
history_lstm = lstm_model.fit(X_train, y_train,
                              epochs=3,
                              batch_size=64,
                              validation_split=0.2)
# Evaluate LSTM
loss_lstm, acc_lstm = lstm_model.evaluate(X_test, y_test)
print("LSTM Accuracy:", acc_lstm)

# GRU MODEL
print("\n--- GRU Model ---")

gru_model = Sequential()
gru_model.add(Embedding(input_dim=5000, output_dim=32, input_length=200))
gru_model.add(GRU(32))
gru_model.add(Dense(1, activation='sigmoid'))

gru_model.compile(optimizer='adam',
                  loss='binary_crossentropy',
                  metrics=['accuracy'])
# Train GRU
history_gru = gru_model.fit(X_train, y_train,
                            epochs=3,
                            batch_size=64,
                            validation_split=0.2)

# Evaluate GRU
loss_gru, acc_gru = gru_model.evaluate(X_test, y_test)
print("GRU Accuracy:", acc_gru)
plt.plot(history_lstm.history['accuracy'], label='LSTM Train Acc')
plt.plot(history_lstm.history['val_accuracy'], label='LSTM Val Acc')

plt.plot(history_gru.history['accuracy'], label='GRU Train Acc')
plt.plot(history_gru.history['val_accuracy'], label='GRU Val Acc')
plt.title("LSTM vs GRU Accuracy")
```

```
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.show()

y_pred_lstm = lstm_model.predict(X_test)
y_pred_gru = gru_model.predict(X_test)

print("\nSample Prediction:")
print("Actual:", y_test[0])
print("LSTM Predicted:", 1 if y_pred_lstm[0] > 0.5 else 0)
print("GRU Predicted:", 1 if y_pred_gru[0] > 0.5 else 0)
```

Output:

```
--- LSTM Model ---
Epoch 1/3
313/313 ━━━━━━━━━━━━━━━━━━━━ 6s 13ms/step - accuracy: 0.7711 - loss: 0.4663 - val_accuracy: 0.8552 - val_loss: 0.3470
Epoch 2/3
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 11ms/step - accuracy: 0.8863 - loss: 0.2852 - val_accuracy: 0.8626 - val_loss: 0.3222
Epoch 3/3
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 11ms/step - accuracy: 0.9092 - loss: 0.2343 - val_accuracy: 0.8596 - val_loss: 0.3242
782/782 ━━━━━━━━━━━━━━━━━━━━ 5s 6ms/step - accuracy: 0.8650 - loss: 0.3214
LSTM Accuracy: 0.8650000095367432

--- GRU Model ---
Epoch 1/3
313/313 ━━━━━━━━━━━━━━━━━━━━ 5s 12ms/step - accuracy: 0.7619 - loss: 0.4749 - val_accuracy: 0.8500 - val_loss: 0.3580
Epoch 2/3
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 11ms/step - accuracy: 0.8853 - loss: 0.2859 - val_accuracy: 0.8668 - val_loss: 0.3194
Epoch 3/3
313/313 ━━━━━━━━━━━━━━━━━━━━ 4s 13ms/step - accuracy: 0.9058 - loss: 0.2415 - val_accuracy: 0.8530 - val_loss: 0.3600
782/782 ━━━━━━━━━━━━━━━━━━━━ 4s 5ms/step - accuracy: 0.8534 - loss: 0.3647
GRU Accuracy: 0.8533999919891357
```




```
Sample Prediction:
Actual: 0
LSTM Predicted: 1
GRU Predicted: 0
```

Experiment Number: 10

Aim: To learn and implement techniques to improve neural network performance and prevent overfitting by experimenting with hyperparameters such as learning rate, batch size, epochs, and optimizers.

Software Used

- Google Colab / Jupyter Notebook / VS Code

Theory

Neural networks may suffer from **overfitting**, where the model performs well on training data but poorly on unseen data.
To improve performance and reduce overfitting, we use:

- **Hyperparameter tuning** (learning rate, batch size, epochs, optimizer)
- **Regularization techniques** (Dropout, Early Stopping)

Methodology

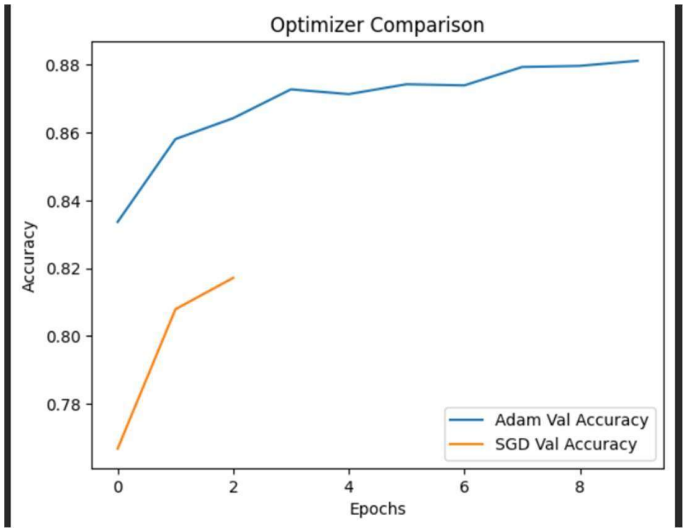
1. Load dataset (Fashion-MNIST)
2. Normalize data
3. Build neural network model
4. Train model with different hyperparameters
5. Apply dropout to prevent overfitting
6. Use early stopping
7. Compare results using graphs

Code:

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import fashion_mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Flatten
from tensorflow.keras.optimizers import Adam, SGD
from tensorflow.keras.callbacks import EarlyStopping

# Step 1: Load dataset
(X_train, y_train), (X_test, y_test) = fashion_mnist.load_data()

# Step 2: Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0
```

```
# Step 3: Build function for model
def create_model(optimizer):
    model = Sequential()
    model.add(Flatten(input_shape=(28,28)))
    model.add(Dense(128, activation='relu'))
    model.add(Dropout(0.3))  # Prevent overfitting
    model.add(Dense(64, activation='relu'))
    model.add(Dense(10, activation='softmax'))

    model.compile(optimizer=optimizer,
                  loss='sparse_categorical_crossentropy',
                  metrics=['accuracy'])
    return model

# Step 4: Early stopping
early_stop = EarlyStopping(monitor='val_loss', patience=3)

# Step 5: Train with Adam optimizer
model_adam = create_model(Adam(learning_rate=0.001))
history_adam = model_adam.fit(X_train, y_train,
                              epochs=10,
                              batch_size=64,
                              validation_split=0.2,
                              callbacks=[early_stop])

# Step 6: Train with SGD optimizer
model_sgd = create_model(SGD(learning_rate=0.01))
history_sgd = model_sgd.fit(X_train, y_train,
                            epochs=10,
                            batch_size=64,
                            validation_split=0.2,
                            callbacks=[early_stop])

# Step 7: Evaluate
loss_adam, acc_adam = model_adam.evaluate(X_test, y_test)
loss_sgd, acc_sgd = model_sgd.evaluate(X_test, y_test)

print("Adam Accuracy:", acc_adam)
print("SGD Accuracy:", acc_sgd)

# Step 8: Graph comparison
```

```
plt.plot(history_adam.history['val_accuracy'], label='Adam Val Accuracy')
plt.plot(history_sgd.history['val_accuracy'], label='SGD Val Accuracy')

plt.title("Optimizer Comparison")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.show()
```

Output:

```
Epoch 1/10
750/750 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.7818 - loss: 0.6147 - val_accuracy: 0.8337 - val_loss: 0.4549
Epoch 2/10
750/750 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.8418 - loss: 0.4393 - val_accuracy: 0.8581 - val_loss: 0.3929
Epoch 3/10
750/750 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.8524 - loss: 0.4029 - val_accuracy: 0.8643 - val_loss: 0.3761
Epoch 4/10
750/750 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.8625 - loss: 0.3762 - val_accuracy: 0.8727 - val_loss: 0.3518
Epoch 5/10
750/750 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.8679 - loss: 0.3595 - val_accuracy: 0.8713 - val_loss: 0.3521
Epoch 6/10
750/750 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.8696 - loss: 0.3512 - val_accuracy: 0.8742 - val_loss: 0.3486
Epoch 7/10
750/750 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.8762 - loss: 0.3353 - val_accuracy: 0.8739 - val_loss: 0.3457
Epoch 8/10
750/750 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.8783 - loss: 0.3296 - val_accuracy: 0.8793 - val_loss: 0.3250
Epoch 9/10
750/750 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.8802 - loss: 0.3183 - val_accuracy: 0.8797 - val_loss: 0.3350
Epoch 10/10
750/750 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.8848 - loss: 0.3117 - val_accuracy: 0.8812 - val_loss: 0.3270
Epoch 1/10
750/750 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - accuracy: 0.6221 - loss: 1.1076 - val_accuracy: 0.7668 - val_loss: 0.6889
Epoch 2/10
750/750 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.7581 - loss: 0.7004 - val_accuracy: 0.8079 - val_loss: 0.5730
Epoch 3/10
750/750 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.7915 - loss: 0.6055 - val_accuracy: 0.8172 - val_loss: 0.5206
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8701 - loss: 0.3562
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8096 - loss: 0.5415
Adam Accuracy: 0.8701000213623047
SGD Accuracy: 0.8095999956130981
```